TCS PREPARATION: 14-DAY 7-PROJECT MASTER CURRICULUM

A Step-by-Step Practical Full-Stack Learning Blueprint & AI Collaboration Guide
Tailored Specifically for TCS Technical Interview Excellence

---

1. GUIDELINES FOR AI ASSISTANT (ANTIGRAVITY)

While guiding the candidate through these 7 projects, the AI assistant MUST follow these 4 teaching rules:

- **Rule 1 (Analogy First):** Before writing code, explain the core concept using a simple real-world analogy (e.g., explaining database foreign keys as student ID cards).
- **Rule 2 (Minute-Detail Code Walkthrough):** Write clean, well-commented code line-by-line and explain what every function, hook, and variable does.
- **Rule 3 (TCS Interview Checkpoints):** At the end of every module, ask 2 sample TCS interview questions on that module and help the candidate practice answering out loud.
- **Rule 4 (Git Hygiene):** Ensure every completed project is committed and pushed to GitHub with a professional README.

2. 14-DAY CURRICULUM OVERVIEW

Day	Project Name	Tech Stack	Key TCS Concept Mastered
Day 1-2	1. DevPortfolio	HTML5, CSS3, JS, Git	DOM, Responsive Flexbox/Grid, Git CLI
Day 3-4	2. WeatherHub	React 19, Tailwind, REST API	useState, useEffect, fetch, Async/Await
Day 5-6	3. TaskPulse	Next.js 16, Supabase SQL	Next App Router, SQL CRUD, Server Components
Day 7-8	4. RoleVault	Next.js, Auth, @supabase/ssr	Session Cookies, RBAC (Admin vs User), Middleware
Day 9-10	5. ShopNova	Next.js, Relational DB, State	Foreign Keys, Relational ERD, Global Cart State
Day 11-12	6. DevSync	Next.js, WebSockets, Storage	Realtime Subscriptions, Cloud File Uploads
Day 13-14	7. AgileBoard	Next.js, DnD, Recharts	Drag & Drop UI state, Analytics Charts
Finale	8. CodeToppers ERP	Full Stack Enterprise Suite	RLS Security, PL/pgSQL RPC, TCS 30s Pitch

PROJECT 1: Interactive Developer Portfolio (dev-portfolio) (Days 1 - 2)

Technologies: HTML5, Vanilla CSS3, JavaScript (ES6+), Git & GitHub Pages

Data Flow: *User Browser -> Index.html -> Styles.css (Flexbox/Grid) -> Script.js (DOM Events / Theme Toggle) -> GitHub Pages Deployment*

Step-by-Step Execution Plan:
1. Build HTML structure with semantic tags (header, section, footer).
2. Apply CSS Flexbox & CSS Grid for mobile-responsive card layout.
3. Add JS DOM listener to toggle dark/light theme dynamically.
4. Initialize Git repository, commit code, and deploy live on GitHub Pages.

TCS Interview Focus: Semantic HTML vs Non-semantic, CSS Box Model, Event Delegation in JS, Basic Git CLI commands (git clone, push, pull, commit).

PROJECT 2: Live Weather & Air Quality Hub (weather-hub-react) (Days 3 - 4)

Technologies: React 19, Tailwind CSS, OpenWeather API / WeatherAPI

Data Flow: *User Inputs City -> React State (useState) -> fetch API (useEffect / async-await) -> Response Parsing -> Dynamic UI Card Render*

Step-by-Step Execution Plan:
1. Create React application using Vite/Next.js.
2. Style search bar and weather card using Tailwind utility classes.
3. Implement useState for tracking search input and weather data.
4. Trigger useEffect and async fetch() on form submit to call live OpenWeather API.
5. Handle loading states and city-not-found error alerts.

TCS Interview Focus: React Virtual DOM, useState vs useEffect, Promises and Async/Await, Difference between HTTP GET and POST.

PROJECT 3: TaskPulse - Next.js Database CRUD App (taskpulse-next-crud) (Days 5 - 6)

Technologies: Next.js 16 (App Router), TypeScript, Supabase (PostgreSQL)

Data Flow: *Browser Form -> Next.js Page -> Supabase JS Client -> PostgreSQL Database (public.tasks table) -> Live Re-render*

Step-by-Step Execution Plan:
1. Create Next.js App Router project.
2. Set up Supabase project & create tasks table with columns: (id, title, status, created_at).
3. Initialize Supabase client in lib/supabase/client.ts.
4. Implement SELECT query to list tasks, INSERT query to add tasks, and DELETE query to remove tasks.
5. Learn difference between Next.js Server Components and Client Components.

TCS Interview Focus: Next.js App Router vs Pages Router, Server Components vs Client Components, SQL CRUD operations, Primary Key constraint.

PROJECT 4: RoleVault - Auth & Role-Based Access Control (rolevault-next-auth) (Days 7 - 8)

Technologies: Next.js 16, Supabase Auth, Cookie Sessions (@supabase/ssr), Tailwind CSS

Data Flow: *Login Screen -> Supabase Auth -> JWT Session Cookie -> Next.js Middleware / Layout -> Role Check (profiles table) -> Protected Page Access*

Step-by-Step Execution Plan:
1. Build Login & Registration forms.
2. Connect Supabase Auth (supabase.auth.signUp & signInWithPassword).
3. Create public.profiles table with role column (admin vs user).
4. Create a protected Dashboard layout that checks user cookies server-side before displaying content.
5. Show Secret Admin Panel only if user role equals admin.

TCS Interview Focus: Authentication vs Authorization, Role-Based Access Control (RBAC), HTTP-only Cookies vs LocalStorage, JWT Token structure.

PROJECT 5: ShopNova - Relational E-Commerce Store (shopnova-ecommerce) (Days 9 - 10)

Technologies: Next.js 16, Supabase PostgreSQL, Zustand / React Context, Tailwind CSS

Data Flow: *Product Catalog -> Category Filter -> Add to Cart (Global State) -> Checkout Form -> PostgreSQL Insert (orders & order_items tables)*

Step-by-Step Execution Plan:
1. Design relational tables: products, categories, orders, order_items with Foreign Key constraints.
2. Build product catalog grid with category filters and search bar.
3. Create Global Cart Context to add/remove items and calculate total prices.
4. Implement checkout form that saves completed orders to PostgreSQL.

TCS Interview Focus: Relational DB entity relationships (One-to-Many, Many-to-Many), Foreign Key constraints, Global State Management.

PROJECT 6: DevSync - Real-Time Chat & Cloud Storage (devsync-realtime-chat) (Days 11 - 12)

Technologies: Next.js 16, Supabase Realtime (WebSockets), Supabase Storage Buckets

Data Flow: *User Sends Message -> PostgreSQL Insert -> Supabase Realtime WebSocket Broadcast -> Live Chat Stream Update without Page Refresh*

Step-by-Step Execution Plan:
1. Create messages table in Supabase.
2. Enable Supabase Realtime WebSockets on messages table.
3. Build live chat UI that listens to incoming message stream.
4. Create Supabase Storage Bucket for file attachments and drag-and-drop uploader.

TCS Interview Focus: WebSockets vs HTTP Polling, Realtime Subscriptions, Cloud Storage buckets, Event-driven architecture.

PROJECT 7: AgileBoard - Jira-Style Issue Tracker & Analytics (agileboard-jira-clone) (Days 13 - 14)

Technologies: Next.js 16, Drag-and-Drop (@hello-pangea/dnd), Recharts, Tailwind CSS

Data Flow: *Interactive Kanban Board (To Do, In Progress, Done) -> Drag & Drop Event -> State Update -> PostgreSQL Sync -> Recharts Analytics Rendering*

Step-by-Step Execution Plan:
1. Build multi-column Kanban board for sprint task management.
2. Implement Drag-and-Drop task card reordering.
3. Integrate Recharts to render task completion analytics and burn-down progress charts.
4. Add export feature for summary reporting.

TCS Interview Focus: Drag and Drop UI state handling, Analytics & Chart visualization libraries, Production Optimization.

GRAND FINALE (PROJECT 8): CodeToppers ERP & CRM MASTERY

After completing Projects 1-7, you will return to your flagship **CodeToppers ERP** project with 100% confidence. You will easily understand every file in d:\codetoppers, refactor leads/page.tsx into clean components, and deliver a flawless 30-second project pitch during your TCS interview!